

CPP102: PROGRAMMING LOGIC AND DESIGN

Week 2 Lecture and Laboratory Module

Tools for Developing Program Logic: Algorithm, Pseudocode, and Flowchart

Department of Computer Engineering

PART A. LECTURE MODULE

1. Module Title

Tools for Developing Program Logic: Algorithm, Pseudocode, and Flowchart

2. Week Number

Week 2

3. Course Title

CPP102 — Programming Logic and Design (2 units, Lecture-Laboratory)

4. Topic Coverage

- Program logic and its role in solving problems
- Algorithms: definition, characteristics, and examples
- Pseudocode: definition, common keywords, and examples
- Flowcharts: definition, standard symbols, and visual representation
- The process of moving from a problem to a working logic design
- Using Mermaid-generated flowcharts as a visualization aid only
- Using Git to track and submit logic design files
- Basic Git commands: git status, git add, git commit, git push

5. Intended Learning Outcomes

By the end of the lesson, students should be able to:

- explain the purpose of algorithms, pseudocode, and flowcharts in program development;
- differentiate algorithm, pseudocode, and flowchart;
- create a simple algorithm for a given programming problem;
- write pseudocode using standard keywords;

- interpret and create basic flowcharts using standard flowchart symbols;
- perform basic Git staging, committing, and pushing.

Mermaid code may be provided only as a tool for generating flowchart visuals. Students are not required to memorize or study Mermaid syntax as a separate lesson.

6. Introduction

When beginners are asked to solve a programming problem, a common mistake is to open the computer and start typing code immediately. This usually leads to confusion, errors, and wasted time.

This week's lesson focuses on three planning tools: the algorithm, the pseudocode, and the flowchart. These tools help a programmer think through the solution before writing a single line of code.

In this module, Mermaid is used only to display flowcharts clearly and neatly. Students do not need to study Mermaid as a programming or scripting language. The important skill is understanding the logic of the flowchart and the correct use of flowchart symbols.

7. Main Lecture Content

a. What is Program Logic?

Program logic refers to the sequence of steps and decisions that a program follows in order to solve a problem and produce a correct result.

Without clear logic, a program may run without errors but still produce the wrong output. Correct logic is what makes a program useful.

Problem-solving comes first, and coding comes after. A programmer studies the problem, identifies the input, determines the process, and specifies the expected output — before writing any code.

b. What is an Algorithm?

An algorithm is a finite set of clear, step-by-step instructions designed to solve a specific problem.

Characteristics of a good algorithm:

- Clear and unambiguous — each step has only one possible meaning.
- Has a definite start and end — it does not run forever.
- Produces a correct, expected output for a given input.
- Consists of a finite number of steps.

Real-life analogy: A recipe for cooking is a good example of an algorithm. It lists ingredients (input), specifies steps (process), and describes the finished dish (output).

c. What is Pseudocode?

Pseudocode is a way of writing the steps of a program using simple, structured, English-like statements, without following the strict syntax of any programming language.

Pseudocode allows a programmer to focus entirely on logic without worrying about syntax errors. It is easier to read than actual code, especially for beginners.

Common pseudocode keywords used in this course:

Keyword	Meaning
START	Marks the beginning of the algorithm
INPUT	Receives a value from the user
PROCESS	Performs a calculation or operation
IF / ELSE	Checks a condition and branches accordingly
DISPLAY	Shows a result or message to the user
END	Marks the end of the algorithm

d. What is a Flowchart?

A flowchart is a diagram that represents the steps of a program using standard symbols connected by arrows. It provides a visual representation of the logic.

Common flowchart symbols:

Symbol Name	Shape	Purpose
Terminal	Oval	Marks the Start or End of the flowchart
Process	Rectangle	Represents a computation or assignment
Input/Output	Parallelogram	Represents data entering or leaving the program
Decision	Diamond	Represents a Yes/No condition
Flowline	Arrow	Shows the direction of the process flow

e. Using Mermaid as a Flowchart Visualization Aid

Mermaid is a text-based diagramming tool that can convert simple text instructions into a flowchart diagram. In this module, Mermaid is used only to help visualize flowcharts neatly.

Students are not expected to master Mermaid syntax. The focus remains on identifying the correct flowchart symbols, arranging the logic correctly, and understanding the flow of the solution.

- Mermaid is used for visualization only — it is not a required programming lesson.
- Students may also create flowcharts using diagrams.net, MS Word shapes, PowerPoint shapes, or by hand if allowed by the instructor.
- Assessment will focus on the correctness of the algorithm, pseudocode, and flowchart logic — not on advanced Mermaid syntax.

To preview a Mermaid flowchart, copy the code and paste it into the free viewer at: <https://mermaid.live>

Note on Git and Mermaid: Since Mermaid code is written as plain text, it can be stored and tracked in Git if needed. However, for this lesson, Git is used mainly to track the submitted activity files. The primary concern is still the correctness of the flowchart logic.

f. Difference Among Algorithm, Pseudocode, and Flowchart

Feature	Algorithm	Pseudocode	Flowchart
Format	Numbered steps	Code-like keywords	Diagram / symbols
Language	Plain English	Structured keywords	Visual symbols
Purpose	Describe logic steps	Prepare for coding	Visualize flow
Audience	Anyone	Programmers	Programmers/teams

g. From Problem to Logic Design

Before a program is coded in Python, a problem passes through these stages:

- Understand the problem — read and clarify exactly what is being asked.
- Identify inputs — determine what data the program needs to receive.
- Identify process — determine what computation or action must be performed.

- Identify output — determine what result the program should display.
- Write algorithm — list the solution as a sequence of plain steps.
- Convert to pseudocode — express the algorithm using structured, code-like statements.
- Convert to flowchart — represent the same logic using standard symbols.
- Convert to Python code — translate the finished logic into actual Python (practiced in later weeks).

h. Introduction to Git for Logic Files

Git is not only useful for tracking Python code. It can already be used at this early planning stage to save and track changes in logic design files such as algorithm text files, pseudocode text files, and flowchart files.

Using Git at this stage builds the habit of saving and documenting work consistently, which is a professional practice in software development.

i. Basic Git Commands for Week 2

Note: Students already set up their Git and GitHub accounts in Week 1. In Week 2, the same basic workflow is applied to logic design files.

Command	What It Does
git status	Shows which files have been added, modified, or are untracked
git add .	Stages all changes in the current folder for the next commit
git commit -m "..."	Saves a snapshot of staged changes with a descriptive message
git push	Uploads committed changes to GitHub

j. Example Git Workflow for Week 2

```
git status
git add .
git commit -m "Add Week 2 problem solving activities"
git push
```

Do not worry about branches, merging, or pull requests at this stage. The focus is on building the habit of staging and committing your work regularly.

8. Key Terms

Term	Definition
Program Logic	The sequence of steps and decisions a program follows to solve a problem
Algorithm	A finite, ordered set of steps designed to solve a specific problem
Pseudocode	Structured English-like statements used to describe logic without strict syntax
Flowchart	A visual diagram using standard symbols to represent the steps and flow of a program
Mermaid	A text-based diagramming tool used in this module only to generate flowchart visuals
git add	Stages files so they are ready to be included in the next commit
git commit	Saves a snapshot of staged changes with a descriptive message
git push	Uploads committed changes to the GitHub remote repository

9. Lecture Examples

The following six examples show how to move from a problem to an algorithm, pseudocode, and flowchart. Mermaid code is provided only to generate a clean visual of the flowchart. Students should focus on the logic and the correct use of flowchart symbols.

For all examples: the Mermaid code is a visualization aid. You are not required to memorize Mermaid syntax. To preview the flowchart, copy the code and paste it into <https://mermaid.live>.

Example 1: Add Two Numbers

Problem:

Ask the user for two numbers, add them, and display the sum.

Algorithm:

Step 1: Start.

Step 2: Ask the user to enter the first number. Store it as num1.

Step 3: Ask the user to enter the second number. Store it as num2.

Step 4: Compute $\text{sum} = \text{num1} + \text{num2}$.

Step 5: Display sum.

Step 6: End.

Pseudocode:

```
START
INPUT num1
INPUT num2
PROCESS sum = num1 + num2
DISPLAY sum
END
```

Flowchart Visualization (Mermaid):

The Mermaid code below is provided only to generate a visual flowchart. Focus on the flowchart logic and symbols — you do not need to memorize the Mermaid syntax.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input num1/]
 B --> C[/Input num2/]
 C --> D[sum = num1 + num2]
 D --> E[/Display sum/]
 E --> F([End])
```
```

Explanation:

This is the simplest type of problem. There is no decision — the program always takes both numbers and adds them. The flow goes straight from Start to End without any branching.

Example 2: Determine if a Student Passed or Failed

Problem:

Ask the user for a student's grade. If the grade is 60 or above, display "Passed". Otherwise, display "Failed".

Algorithm:

Step 1: Start.

Step 2: Ask the user to enter the grade. Store it as grade.

Step 3: Check if grade is 60 or above.

Step 4: If yes, display "Passed".

Step 5: If no, display "Failed".

Step 6: End.

Pseudocode:

```
START
INPUT grade
IF grade >= 60 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
```

Flowchart Visualization (Mermaid):

The Mermaid code below is provided only to generate a visual flowchart. Focus on the flowchart logic and symbols — you do not need to memorize the Mermaid syntax.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input grade/]
 B --> C{Is grade >= 60?}
 C -- Yes --> D[/Display Passed/]
 C -- No --> E[/Display Failed/]
 D --> F([End])
 E --> F
```
```

Explanation:

This problem introduces a decision. The diamond symbol shows where the program chooses between two paths. Both paths eventually lead to the End.

Example 3: Determine if a Number is Positive or Negative

Problem:

Ask the user for a number. If the number is 0 or above, display "Positive". Otherwise, display "Negative".

Algorithm:

Step 1: Start.

Step 2: Ask the user to enter a number. Store it as number.

Step 3: Check if number is greater than or equal to 0.

Step 4: If yes, display "Positive".

Step 5: If no, display "Negative".

Step 6: End.

Pseudocode:

```
START
INPUT number
IF number >= 0 THEN
    DISPLAY "Positive"
ELSE
    DISPLAY "Negative"
END IF
END
```

Flowchart Visualization (Mermaid):

The Mermaid code below is provided only to generate a visual flowchart. Focus on the flowchart logic and symbols — you do not need to memorize the Mermaid syntax.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input number/]
 B --> C{Is number >= 0?}
 C -- Yes --> D[/Display Positive/]
 C -- No --> E[/Display Negative/]
 D --> F([End])
 E --> F
```
```

Explanation:

The structure of this flowchart is identical to Example 2. Once you understand the Yes/No decision pattern, you can apply it to many different problems.

Example 4: Compute the Average of Three Grades

Problem:

Ask the user for three grades, compute the average, and display the result.

Algorithm:

Step 1: Start.

Step 2: Ask the user to enter the first grade. Store it as grade1.

Step 3: Ask the user to enter the second grade. Store it as grade2.

Step 4: Ask the user to enter the third grade. Store it as grade3.

Step 5: Compute average = (grade1 + grade2 + grade3) / 3.

Step 6: Display average.

Step 7: End.

Pseudocode:

```
START
INPUT grade1
INPUT grade2
INPUT grade3
PROCESS average = (grade1 + grade2 + grade3) / 3
DISPLAY average
END
```

Flowchart Visualization (Mermaid):

The Mermaid code below is provided only to generate a visual flowchart. Focus on the flowchart logic and symbols — you do not need to memorize the Mermaid syntax.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input grade1/]
 B --> C[/Input grade2/]
 C --> D[/Input grade3/]
 D --> E[average = (grade1+grade2+grade3)/3]
 E --> F[/Display average/]
 F --> G([End])
```
```

Explanation:

This example involves three inputs followed by one computation. There is no decision — the flow is linear from Start to End. The formula is written inside the process rectangle.

Example 5: Determine if a Number is Even or Odd

Problem:

Ask the user for a number. If the number is divisible by 2, display "Even". Otherwise, display "Odd".

Algorithm:

Step 1: Start.

Step 2: Ask the user to enter a number. Store it as number.

Step 3: Check if number MOD 2 equals 0.

Step 4: If yes, display "Even".

Step 5: If no, display "Odd".

Step 6: End.

Pseudocode:

```
START
INPUT number
IF number MOD 2 = 0 THEN
    DISPLAY "Even"
ELSE
    DISPLAY "Odd"
END IF
END
```

Flowchart Visualization (Mermaid):

The Mermaid code below is provided only to generate a visual flowchart. Focus on the flowchart logic and symbols — you do not need to memorize the Mermaid syntax.

```
```mermaid
graph TD
 A([Start]) --> B[/Input number/]
 B --> C{Is number MOD 2 = 0?}
 C -- Yes --> D[/Display Even/]
 C -- No --> E[/Display Odd/]
 D --> F([End])
 E --> F
```
```

Explanation:

MOD means the remainder after division. If a number divided by 2 has a remainder of 0, the number is even. This is a very common pattern in programming logic.

Example 6: Compute the Area of a Rectangle**Problem:**

Ask the user for the length and width of a rectangle. Compute the area and display the result.

Algorithm:

Step 1: Start.

Step 2: Ask the user to enter the length. Store it as length.

Step 3: Ask the user to enter the width. Store it as width.

Step 4: Compute area = length * width.

Step 5: Display area.

Step 6: End.

Pseudocode:

```
START
INPUT length
INPUT width
PROCESS area = length * width
DISPLAY area
END
```

Flowchart Visualization (Mermaid):

The Mermaid code below is provided only to generate a visual flowchart. Focus on the flowchart logic and symbols — you do not need to memorize the Mermaid syntax.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input length/]
 B --> C[/Input width/]
 C --> D[area = length * width]
 D --> E[/Display area/]
 E --> F([End])
```
```

Explanation:

This is a straightforward computation problem with two inputs and one output. The logic is linear — no decisions are needed.

10. Student Checkpoint Questions

(To be asked briefly during the lecture to check understanding. Answers are in Part D.)

- What is program logic?

- What is an algorithm?
- Give one characteristic of a good algorithm.
- What is pseudocode, and why is it useful before coding?
- Give two common pseudocode keywords.
- What is a flowchart?
- What flowchart symbol is used to represent a decision?
- What is the difference between an algorithm and pseudocode?
- What is the purpose of git add?
- What is the purpose of git commit?

11. Lecture Summary

Before writing actual Python code, programmers first plan the logic of their solution. An algorithm presents this logic as numbered steps in plain language. Pseudocode expresses the same logic using structured, code-like keywords. A flowchart shows the same logic as a visual diagram using standard symbols.

In this module, Mermaid is used only as a convenience tool to display flowcharts clearly. It is not a required topic or a skill that will be assessed on its own. Students may also draw flowcharts using diagrams.net, MS Word, PowerPoint, or by hand.

The Git workflow for this week is: git status → git add → git commit → git push. Students should practice this workflow every time they save or update a logic design file.

PART B. LABORATORY MODULE

1. Laboratory Title

Creating Algorithms, Pseudocode, and Flowcharts with Git Commit

2. Laboratory Objectives

By the end of the laboratory activity, students should be able to:

- analyze a simple programming problem;
- identify the input, process, and output of a problem;
- create an algorithm;
- write pseudocode;
- create a flowchart using standard symbols;
- save their work in a properly named project folder;
- use git status, git add, git commit, and git push.

3. Required Materials

- Computer or laptop
- Git installed
- GitHub account
- Text editor or VS Code
- Any flowchart tool: diagrams.net, MS Word shapes, PowerPoint shapes, or <https://mermaid.live> for Mermaid-generated flowcharts

4. Pre-Lab Discussion

In this laboratory activity, students will practice transforming three different problems into algorithms, pseudocode, and flowcharts. Students already set up their Git and GitHub accounts in Week 1. In Week 2, the same workflow is applied to logic design files.

Flowcharts may be submitted as Mermaid-generated diagrams, diagrams.net exports, MS Word/PowerPoint shape drawings, or hand-drawn diagrams if allowed by the instructor. If Mermaid is used, students may copy the structure from the lecture examples and modify the labels. The focus is not on Mermaid coding but on correct flowchart logic.

5. Laboratory Activity Proper

Laboratory Activity 1: Add Two Numbers

Problem: "Create a logic design that asks for two numbers, adds them, and displays the sum."

Step 1: Input-Process-Output Table

Fill in the table below before writing your algorithm.

| INPUT | PROCESS | OUTPUT |
|--------------|-------------------|--------|
| num1
num2 | sum = num1 + num2 | sum |

Step 2: Create an Algorithm

Write a numbered, step-by-step algorithm for this problem.

Step 1: ... Step 2: ... Step 3: ...

Step 3: Write Pseudocode

Convert your algorithm to pseudocode using the keywords: START, INPUT, PROCESS, DISPLAY, END.

START INPUT

Step 4: Create a Flowchart

Create a flowchart for this problem using any instructor-approved format: Mermaid, diagrams.net, MS Word shapes, PowerPoint shapes, or hand drawing. Your flowchart must use correct flowchart symbols and show the correct logic.

Attach your flowchart here, or write Mermaid code below.

Laboratory Activity 2: Compute the Average of Three Quiz Scores

Problem: "Create a logic design that asks for three quiz scores, computes the average, and displays the result."

Step 1: Input-Process-Output Table

| INPUT | PROCESS | OUTPUT |
|----------------------------|---------------------------------------|---------|
| score1
score2
score3 | average =
(score1+score2+score3)/3 | average |

Step 2: Create an Algorithm

Step 1: ... Step 2: ... Step 3: ...

Step 3: Write Pseudocode

START INPUT

Step 4: Create a Flowchart

Create a flowchart for this problem using any instructor-approved format.

Attach your flowchart here, or write Mermaid code below.

Laboratory Activity 3: Determine if a Number is Even or Odd

Problem: "Create a logic design that asks for a number and displays whether the number is even or odd."

Step 1: Input-Process-Output Table

| INPUT | PROCESS | OUTPUT |
|--------|-----------------------------|-----------------|
| number | Check: is number MOD 2 = 0? | "Even" or "Odd" |

Step 2: Create an Algorithm

Step 1: ... Step 2: ... Step 3: ...

Step 3: Write Pseudocode

START INPUT

Step 4: Create a Flowchart

This problem requires a decision step. Your flowchart must include a decision diamond with Yes and No branches.

Attach your flowchart here, or write Mermaid code below.

6. Saving Files and Git Commit

Create a folder named: cpp102-week2-lastname (replace lastname with your actual last name).

Inside the folder, save the following files:

| Required File | Description |
|------------------------|--|
| algorithm.txt | All three algorithms |
| pseudocode.txt | All three pseudocode solutions |
| flowchart1.md or image | Flowchart for Activity 1 (any approved format) |
| flowchart2.md or image | Flowchart for Activity 2 (any approved format) |
| flowchart3.md or image | Flowchart for Activity 3 (any approved format) |
| README.md | Name, section, course, and short reflection |

README.md Format:

```
Name: [Your Full Name]
Section: [Your Section]
Course: CPP102 – Programming Logic and Design
Week 2 Activity: Algorithm, Pseudocode, and Flowchart
Activities Completed: Activity 1, Activity 2, Activity 3
```

```
Short Reflection:
[Write 2-3 sentences about what you learned from this activity.]
```

Git Workflow:

```
git status
git add .
git commit -m "Add Week 2 algorithm pseudocode and flowchart
activities"
git push
```

Take a screenshot of your terminal after running git status and after a successful git commit. Include the screenshots in your submission.

7. Expected Output

Students should submit the following:

- algorithm.txt
- pseudocode.txt
- flowchart1, flowchart2, and flowchart3 (any instructor-approved format)

- README.md with name, section, course, activities completed, and short reflection
- Screenshot of git status
- Screenshot of successful git commit
- GitHub repository link, if required by the instructor

8. Rubric (100 points)

| Criteria | Full Marks | Partial | Score |
|---|------------|---------|-------|
| Input-Process-Output Analysis (all 3 activities) | 15 | 7 | |
| Algorithm Correctness and Clarity (all 3 activities) | 20 | 10 | |
| Pseudocode Correctness and Readability (all 3 activities) | 20 | 10 | |
| Flowchart Correctness and Symbol Usage (all 3 activities) | 25 | 12 | |
| Git Commit Evidence and File Organization | 10 | 5 | |
| Completeness, Neatness, and README.md Reflection | 10 | 5 | |
| TOTAL | 100 | | |

PART C. ASSESSMENT TASK

Lab Exercise 2: Algorithm, Pseudocode, and Flowchart

Instructions to Students

Solve each of the 10 problems below. For every problem, you must provide:

- A. Algorithm — a numbered, step-by-step description of the solution in plain language.
- B. Pseudocode — using the keywords: START, INPUT, PROCESS, IF, ELSE, DISPLAY, END.
- C. Flowchart — a diagram using correct standard flowchart symbols.

The flowchart may be created using Mermaid, diagrams.net, MS Word shapes, PowerPoint shapes, or hand drawing if allowed by the instructor. Mermaid code is acceptable only as a visualization format. The score will be based on the correctness of the flowchart logic, symbols, and flow direction — not on advanced Mermaid syntax.

File organization:

- Create a folder named: cpp102-week2-lastname
- File names: problem1_algorithm.txt, problem1_pseudocode.txt, problem1_flowchart.md (or image)
- Include a README.md with your name, section, course, and a short reflection.
- After saving all files, run: git status → git add . → git commit -m "Add Week 2 assessment problems" → git push

Assessment Problems

For each problem, provide the Algorithm, Pseudocode, and Flowchart.

1. Add two numbers and display the sum.
2. Subtract two numbers and display the difference.
3. Multiply two numbers and display the product.
4. Compute the average of three grades and display the result.
5. Compute the area of a rectangle (area = length × width) and display the result.
6. Compute the perimeter of a square (perimeter = 4 × side) and display the result.
7. Convert minutes to seconds (seconds = minutes × 60) and display the result.
8. Ask for a student's grade. Display "Passed" if the grade is 60 or above; otherwise, display "Failed".
9. Ask for a number. Display "Positive" if the number is 0 or above; otherwise, display "Negative".

10. Ask for a number. Display "Even" if the number is divisible by 2; otherwise, display "Odd".

20-Item Formative Quiz: Algorithm, Pseudocode, Flowchart, and Git

Answer key is provided separately in Part D.

I. Multiple Choice (Items 1–8)

1. Which of the following best defines an algorithm?
 - A. A diagram showing program flow
 - B. A finite set of step-by-step instructions to solve a problem
 - C. A type of programming language
 - D. A Git command
2. Pseudocode is best described as:
 - A. The exact syntax of Python
 - B. A structured, code-like way of writing program steps in plain language
 - C. A type of flowchart symbol
 - D. A Git staging command
3. Which flowchart symbol represents a decision point?
 - A. Rectangle
 - B. Oval
 - C. Diamond
 - D. Parallelogram
4. Which flowchart symbol represents the start or end of a flowchart?
 - A. Terminal/Oval
 - B. Process/Rectangle
 - C. Decision/Diamond
 - D. Flowline/Arrow
5. Which pseudocode keyword is used to show output to the user?
 - A. INPUT
 - B. PROCESS
 - C. DISPLAY
 - D. START
6. What is the correct order of the logic design process?
 - A. Pseudocode → Algorithm → Flowchart → Code
 - B. Flowchart → Algorithm → Pseudocode → Code
 - C. Algorithm → Pseudocode → Flowchart → Code
 - D. Code → Algorithm → Pseudocode → Flowchart

7. Which Git command stages a file for the next commit?

A. git status B. git add C. git commit D. git push

8. Which Git command saves a snapshot of the staged changes with a message?

A. git status B. git add C. git commit D. git clone

II. True or False (Items 9–14)

9. An algorithm must always be written using flowchart symbols.

10. Pseudocode does not need to follow the strict syntax rules of a programming language.

11. A decision symbol in a flowchart is shaped like a rectangle.

12. git add must be performed before git commit in order for changes to be saved.

13. Flowcharts help a programmer visualize the logic and flow of a program.

14. git push uploads committed changes to GitHub.

III. Identification (Items 15–20)

15. The sequence of steps and decisions a program follows to solve a problem.

16. The pseudocode keyword used to receive a value from the user.

17. The flowchart symbol shaped like a parallelogram, representing data entering or leaving the program.

18. The arrow or line connecting flowchart symbols, showing the direction of the flow.

19. The Git command that shows the current state of files in a project folder.

20. A short, descriptive note attached to a Git commit, explaining what was added or changed.

PART D. ANSWER KEY

(For instructor use. This section may be omitted when printing the student copy.)

1. Answers to Lecture Checkpoint Questions

1. Program logic is the sequence of steps and decisions a program follows to solve a problem and produce a correct result.
2. An algorithm is a finite set of clear, step-by-step instructions used to solve a specific problem.
3. Any correct answer, such as: it is clear and unambiguous, has a definite start and end, or produces a correct output for a given input.
4. Pseudocode is a way of writing program steps using simple, structured, English-like statements; it is useful before coding because it allows the programmer to focus on logic without worrying about syntax errors.
5. Any two of: START, INPUT, PROCESS, IF, ELSE, DISPLAY, END.
6. A flowchart is a diagram that uses standard symbols and arrows to represent the steps and flow of a program.
7. The diamond (decision) symbol.
8. An algorithm is written in plain, numbered steps; pseudocode is written using structured, code-like keywords such as START, INPUT, DISPLAY, and END.
9. git add stages or prepares a file so it will be included in the next commit.
10. git commit saves a snapshot of the staged changes, along with a descriptive message.

2. Suggested Answers for Lecture Examples

The complete algorithm, pseudocode, and flowchart visualization for all 6 lecture examples are already provided in Part A, Section 9. Students should compare their work against those examples.

3. Suggested Answers for Laboratory Activities

Laboratory Activity 1: Add Two Numbers

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for num1.

Step 3: Ask for num2.

Step 4: Compute $\text{sum} = \text{num1} + \text{num2}$.

Step 5: Display sum.

Step 6: End.

Suggested Pseudocode:

```
START
INPUT num1
INPUT num2
PROCESS sum = num1 + num2
DISPLAY sum
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input num1/]
 B --> C[/Input num2/]
 C --> D[sum = num1 + num2]
 D --> E[/Display sum/]
 E --> F([End])
```
```

Laboratory Activity 2: Compute the Average of Three Quiz Scores

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for score1.

Step 3: Ask for score2.

Step 4: Ask for score3.

Step 5: Compute average = (score1 + score2 + score3) / 3.

Step 6: Display average.

Step 7: End.

Suggested Pseudocode:

```
START
INPUT score1
INPUT score2
INPUT score3
```

```
PROCESS average = (score1+score2+score3)/3
DISPLAY average
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input score1/]
 B --> C[/Input score2/]
 C --> D[/Input score3/]
 D --> E[average = (score1+score2+score3)/3]
 E --> F[/Display average/]
 F --> G([End])
```
```

Laboratory Activity 3: Determine if a Number is Even or Odd

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for number.

Step 3: Check if number MOD 2 equals 0.

Step 4: If yes, display "Even".

Step 5: If no, display "Odd".

Step 6: End.

Suggested Pseudocode:

```
START
INPUT number
IF number MOD 2 = 0 THEN
    DISPLAY "Even"
ELSE
    DISPLAY "Odd"
END IF
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input number/]
 B --> C{Is number MOD 2 = 0?}
 C -- Yes --> D[/Display Even/]
 C -- No --> E[/Display Odd/]
 D --> F([End])
 E --> F
```
```

4. Suggested Answers for Assessment Problems

Problem 1: Add Two Numbers

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for num1.

Step 3: Ask for num2.

Step 4: Compute $\text{sum} = \text{num1} + \text{num2}$.

Step 5: Display sum.

Step 6: End.

Suggested Pseudocode:

```
START
INPUT num1
INPUT num2
PROCESS sum = num1 + num2
DISPLAY sum
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input num1/]
 B --> C[/Input num2/]
 C --> D[sum = num1 + num2]
 D --> E[/Display sum/]
 E --> F([End])
```
```

Problem 2: Subtract Two Numbers

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for num1.

Step 3: Ask for num2.

Step 4: Compute difference = num1 - num2.

Step 5: Display difference.

Step 6: End.

Suggested Pseudocode:

```
START
INPUT num1
INPUT num2
PROCESS difference = num1 - num2
DISPLAY difference
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input num1/]
 B --> C[/Input num2/]
 C --> D[difference = num1 - num2]
 D --> E[/Display difference/]
 E --> F([End])
```
```

Problem 3: Multiply Two Numbers

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for num1.

Step 3: Ask for num2.

Step 4: Compute product = num1 * num2.

Step 5: Display product.

Step 6: End.

Suggested Pseudocode:

```

START
INPUT num1
INPUT num2
PROCESS product = num1 * num2
DISPLAY product
END
  
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```

```mermaid
flowchart TD
 A([Start]) --> B[/Input num1/]
 B --> C[/Input num2/]
 C --> D[product = num1 * num2]
 D --> E[/Display product/]
 E --> F([End])
  ```
  
```

Problem 4: Compute the Average of Three Grades

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for grade1.

Step 3: Ask for grade2.

Step 4: Ask for grade3.

Step 5: Compute average = (grade1 + grade2 + grade3) / 3.

Step 6: Display average.

Step 7: End.

Suggested Pseudocode:

```

START
INPUT grade1
INPUT grade2
INPUT grade3
  
```

```
PROCESS average = (grade1+grade2+grade3)/3
DISPLAY average
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input grade1/]
 B --> C[/Input grade2/]
 C --> D[/Input grade3/]
 D --> E[average = (grade1+grade2+grade3)/3]
 E --> F[/Display average/]
 F --> G([End])
```
```

Problem 5: Compute the Area of a Rectangle**Suggested Algorithm:**

Step 1: Start.

Step 2: Ask for length.

Step 3: Ask for width.

Step 4: Compute area = length * width.

Step 5: Display area.

Step 6: End.

Suggested Pseudocode:

```
START
INPUT length
INPUT width
PROCESS area = length * width
DISPLAY area
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
```

```

flowchart TD
 A([Start]) --> B[/Input length/]
 B --> C[/Input width/]
 C --> D[area = length * width]
 D --> E[/Display area/]
 E --> F([End])
 ...

```

## Problem 6: Compute the Perimeter of a Square

### Suggested Algorithm:

Step 1: Start.

Step 2: Ask for side.

Step 3: Compute perimeter = 4 \* side.

Step 4: Display perimeter.

Step 5: End.

### Suggested Pseudocode:

```

START
INPUT side
PROCESS perimeter = 4 * side
DISPLAY perimeter
END

```

### Suggested Flowchart Visualization (Mermaid):

*The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.*

```

```mermaid
flowchart TD
    A([Start]) --> B[/Input side/]
    B --> C[perimeter = 4 * side]
    C --> D[/Display perimeter/]
    D --> E([End])
    ...
  
```

Problem 7: Convert Minutes to Seconds

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for minutes.

Step 3: Compute $\text{seconds} = \text{minutes} * 60$.

Step 4: Display seconds.

Step 5: End.

Suggested Pseudocode:

```
START
INPUT minutes
PROCESS seconds = minutes * 60
DISPLAY seconds
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input minutes/]
 B --> C[seconds = minutes * 60]
 C --> D[/Display seconds/]
 D --> E([End])
```
```

Problem 8: Passed or Failed

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for grade.

Step 3: Check if grade is 60 or above.

Step 4: If yes, display "Passed".

Step 5: If no, display "Failed".

Step 6: End.

Suggested Pseudocode:

```
START
INPUT grade
IF grade >= 60 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
```

```
END IF
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input grade/]
 B --> C{Is grade >= 60?}
 C -- Yes --> D[/Display Passed/]
 C -- No --> E[/Display Failed/]
 D --> F([End])
 E --> F
```
```

Problem 9: Positive or Negative

Suggested Algorithm:

Step 1: Start.

Step 2: Ask for number.

Step 3: Check if number is 0 or above.

Step 4: If yes, display "Positive".

Step 5: If no, display "Negative".

Step 6: End.

Suggested Pseudocode:

```
START
INPUT number
IF number >= 0 THEN
    DISPLAY "Positive"
ELSE
    DISPLAY "Negative"
END IF
END
```

Suggested Flowchart Visualization (Mermaid):

The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.

```
```mermaid
flowchart TD
 A([Start]) --> B[/Input number/]
 B --> C{Is number >= 0?}
 C -- Yes --> D[/Display Positive/]
 C -- No --> E[/Display Negative/]
 D --> F([End])
 E --> F
```

## Problem 10: Even or Odd

### Suggested Algorithm:

Step 1: Start.

Step 2: Ask for number.

Step 3: Check if number MOD 2 equals 0.

Step 4: If yes, display "Even".

Step 5: If no, display "Odd".

Step 6: End.

### Suggested Pseudocode:

```
START
INPUT number
IF number MOD 2 = 0 THEN
 DISPLAY "Even"
ELSE
 DISPLAY "Odd"
END IF
END
```

### Suggested Flowchart Visualization (Mermaid):

*The Mermaid code is provided for visualization only. Equivalent flowcharts drawn using standard symbols — by hand, diagrams.net, MS Word, or PowerPoint — should also be considered correct.*

```
```mermaid
flowchart TD
    A([Start]) --> B[/Input number/]
    B --> C{Is number MOD 2 = 0?}
    C -- Yes --> D[/Display Even/]
    C -- No --> E[/Display Odd/]
    D --> F([End])
    E --> F
```

....

5. Answer Key for the 20-Item Formative Quiz

Multiple Choice:

1. B 2. B 3. C 4. A 5. C 6. C 7. B 8. C

True or False:

9. False 10. True 11. False 12. True 13. True 14. True

Identification:

15. Program logic

16. INPUT

17. Input/Output symbol (parallelogram)

18. Flowline (arrow)

19. git status

20. Commit message

PART E. REFERENCES

Students are encouraged to use the listed textbooks as primary references. Online materials are supplementary.

Textbooks and Library References

Pomperada, J. R., et al. (2019). Fundamentals of Python Programming. Mindshapes Co., Inc.

Abante, M. V., et al. (2019). Python Programming. Unlimited Books Library Services & Publishing Inc.

Hudson, D. (2020). Python Programming for Beginners. Larsen & Keller.

Online References

Python Software Foundation. (n.d.). Python documentation. <https://docs.python.org/3/>. Accessed June 2026.

Git. (n.d.). Git documentation. <https://git-scm.com/doc>. Accessed June 2026.

GitHub, Inc. (n.d.). GitHub Docs. <https://docs.github.com/en/get-started>. Accessed June 2026.

W3Schools. (n.d.). Python tutorial. <https://www.w3schools.com/python/>. Accessed June 2026.

TutorialsPoint. (n.d.). Python tutorial. <https://www.tutorialspoint.com/python3/index.htm>. Accessed June 2026.

diagrams.net. (n.d.). Flowchart maker. <https://www.diagrams.net/>. Accessed June 2026.

Mermaid. (n.d.). Mermaid live editor. <https://mermaid.live>. Accessed June 2026.